

Logs que a IA entende

Observabilidade em Node.js com Pino + OpenTelemetry

Esquenta **JSConf Brasil 2026**

Quem tá falando? 🙋

Wellington Santana

CTO & Co-founder @ **Kodus**

- **Code review com IA** — open source e sem lock-in de modelo
- Levando uma **devtool brasileira** pro mapa global 🌐

3h da manhã.

Alerta dispara.

Você abre o log e vê:

deu erro

Boa sorte. 🙄

Pra quem você escreve o log?

Log bom **pra humano** já ajuda no dia a dia.

Mas log bom **pra máquina** — suas ferramentas, automações e IA —
troca "*ler log*" por "*fazer pergunta*".

“

Log não é pra você ler. É pra você perguntar.

”

Logs que não ajudam ninguém

```
console.log('entrou aqui');           // entrou onde? quando? quem?  
console.log('erro!!!');               // qual erro? de quê?  
console.log(user);                   // [object Object] 🙄  
console.log('ok', 200, true, payload); // sem chave, sem schema  
logger.info('processando...');        // e aí? terminou? deu certo?  
console.log(JSON.stringify(req));     // 2KB de ruído + vaza PII
```

Todos rodam liso. **Nenhum responde uma pergunta às 3h da manhã.**

Por que se preocupar CEDO

- Log ruim = **dívida que cobra juros em todo incidente**
- MTTR alto = dinheiro + reputação queimando
- Retrofit é caro: schema, PII, trace — tudo dói com a casa cheia
- O investimento certo é **minúsculo**: 1 logger, 1 middleware, 1 schema

“ Observabilidade é decisão de **arquitetura**, não de última hora. ”

Os 3 sinais do seu sistema

Toda aplicação em produção emite 3 tipos de sinal. Eles se completam:

Métrica te acorda — *algo está errado*

Trace te localiza — *onde no caminho quebrou*

Log te explica — *o que exatamente aconteceu*

“

Métrica diz que tem fogo. Trace diz em qual cômodo.

Log diz o que pegou fogo.

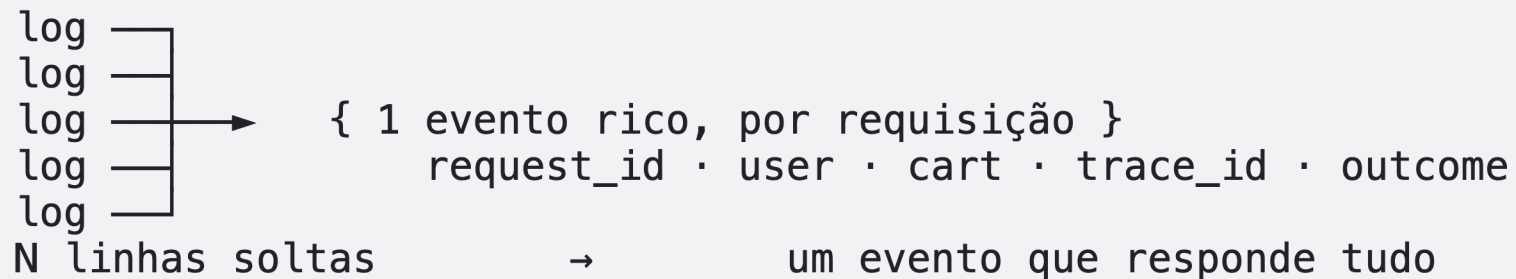
”

Log x Trace x Métrica

	Log	Trace	Métrica
Responde	o quê	onde	quanto
Unidade	1 evento	1 requisição (spans)	1 número agregado
Exemplo	INVALID_AMOUNT, premium	auth→cart→pay, 37ms	erro = 2,3% / 5min
Melhor pra	causa raiz, negócio	latência, gargalo	alerta, tendência

E o que **costura** os três? Um id: o **trace_id** está no log e no trace.

Wide Events — o conceito central



A **Stripe** chama de *canonical log lines*; a **Honeycomb**, de *wide events*.

“ **Mesmo conceito: um evento, não dez linhas soltas.** ”

O que entra no evento

Quanto mais rico, mais perguntas ele responde **sem você ter previsto:**

Tipo	Exemplos	Pra quê
Alta cardinalidade	<code>request_id</code> , <code>user_id</code> , <code>trace_id</code>	achar UMA agulha
Alta dimensionalidade	muitos campos	responder o que você nem perguntou
Contexto de negócio	plano, valor, cupom	<i>"premium não pagou R\$ 49,90"</i>
Contexto de ambiente	<code>commit</code> , <code>version</code> , <code>region</code>	correlacionar com deploy

Os princípios de um log que serve

Construir log bom não é sorte — são **4 decisões**. O resto do deck é detalhe disso:

1. **Estrutura** — JSON, não texto livre
2. **Um evento por requisição** — wide event, não N linhas soltas
3. **Contexto** — negócio (quem, quanto) + ambiente (commit, region)
4. **Correlação** — `trace_id` ligando log ↔ trace

“ Estrutura, contexto e correlação. O resto é ferramenta. ”

O que é o Pino?

Uma biblioteca de log focada em **velocidade** e **JSON**.

- ⚡ **Rápido** — escreve JSON e sai do caminho; overhead baixíssimo no event loop
- 🧱 **Estruturado por padrão** — você loga **objetos**, não frases soltas
- 🔌 **Onipresente** — o Fastify usa por baixo; integra com Nest, Express...
- 🛡️ **Redaction nativa** — mascara senha/cartão/token sem biblioteca extra
- 🚚 **Transports** — manda pra arquivo, Loki, Elastic... **fora** do hot path

“

Faz o básico muito bem e barato: **JSON rápido no stdout**.

”

Pino: o logger único

```
const logger = pino({
  base: { service, version, commit, env, region }, // ambiente em TODO log
  timestamp: pino.stdTimeFunctions.isoTime,
  formatters: { level: (l) => ({ level: l }) }, // "info", não 30
  redact: { paths: ['req.headers.authorization', '*.password', '*.token',
    '*.cardNumber'], censor: '[REDACTED]' }, // 🔒 segurança
  mixin() { /* injeta trace_id do OpenTelemetry */ },
});
```

Rápido, JSON, padrão do ecossistema.

O middleware: 1 evento por requisição

```
function wideEvent(req, res, next) {  
  const start = Date.now();  
  req.wideEvent = { request_id: randomUUID(), method: req.method, path: req.path };  
  
  res.on('finish', () => { // dispara no fim da requisição  
    req.wideEvent.status_code = res.statusCode;  
    req.wideEvent.duration_ms = Date.now() - start;  
    logger.info(req.wideEvent, 'request completed'); // emite UMA vez  
  });  
  
  next();  
}
```

Handler só adiciona **contexto de negócio**. A infra é do middleware.

O handler — só negócio

```
app.post('/checkout', async (req, res) => {
  const ev    = req.wideEvent;           // criado no middleware
  const user  = getUser(req.body.userId);
  const charge = await payment.charge(cart);

  // o handler só enriquece o evento com NEGÓCIO:
  ev.user      = { id: user.id, subscription: user.subscription };
  ev.cart      = { subtotal_cents: cart.subtotal, total_cents: cart.total };
  ev.payment   = { id: charge.id, status: charge.status };
  ev.event     = 'checkout.completed';   // nome canônico: domain.action
  ev.outcome   = 'success';

  res.json({ ok: true });
});
```

Zero `console.log`. Só **negócio** indo pro evento.

...e o evento que sai

```
{
  "event": "checkout.failed", "level": "error",
  "service": "checkout-service", "version": "2.3.1",
  "commit": "a1b2c3d", "region": "sa-east-1",
  "trace_id": "9d93b0f7...", "request_id": "0c3a2151-...",
  "user": { "id": "u_1001", "subscription": "premium" },
  "coupon": { "code": "SUPER90", "discount_cents": 9000 },
  "cart": { "subtotal_cents": 4990, "total_cents": -4010 },
  "error": { "code": "INVALID_AMOUNT", "stack": "...at charge (payment.js:16)" },
  "outcome": "error", "duration_ms": 36.88
}
```

Uma linha com nome de evento (`domain.action`): quem, o quê, por quê, onde.

O que é OpenTelemetry?

O **padrão aberto** (CNCF) pra gerar logs, métricas e traces.

Antes: cada ferramenta tinha seu SDK. Trocar de fornecedor = reinstrumentar tudo.

Agora: instrumenta **uma vez**, exporta pra qualquer backend.

-  **Sem lock-in** — Datadog hoje, Grafana amanhã; a app não muda
-  **Auto-instrumentation** — http, Express, banco viram trace **sozinhos**
-  **Propagação** — o mesmo `trace_id` cruza todos os seus serviços
-  **Virou padrão de mercado** — todo grande fornecedor fala OTel hoje

Anatomia de um trace

```
trace_id: 9d93b0f7... (a requisição inteira)
└─ span: POST /checkout [0-37ms] (span raiz)
    └─ span: middleware [0-1ms]
    └─ span: getUser [1-2ms]
    └─ span: payment.charge [2-37ms] ← o gargalo
```

- **Trace** = a jornada · **Span** = uma etapa (início, fim, duração)
- **Propagação** = o `trace_id` viaja entre serviços (header `traceparent`)
- O OpenTelemetry gera esses spans **automaticamente** — você não escreve nenhum à mão

OpenTelemetry: log ↔ trace

- Log diz **o quê**. Trace diz **onde no caminho**.
- Carregado **antes de tudo** (`-r ./tracing.js`) → instrumenta http/express
- Auto-instrumentation → `trace_id` aparece **sozinho** no log
- Mesmo `trace_id` no **log** e no **trace** → pulo de um clique

```
mixin() {  
  const span = trace.getSpan(context.active());  
  return span ? { trace_id: span.spanContext().traceId } : {};  
}
```

“ Auto-instrumentation te dá o **trace** de graça. O wide event é **você instrumentando o log** com o negócio. ”

Segurança / LGPD — redaction

```
curl -XPOST /login -H 'authorization: Bearer super-secret' \  
-d '{"password":"123456"}'
```

```
"req": { "headers": { "authorization": "[REDACTED]" } },  
"credentials": { "password": "[REDACTED]" },  
"payment_method": { "cardNumber": "[REDACTED]", "token": "[REDACTED]" }
```

Redaction na origem. Custo ~zero. **Log também é dado sensível: LGPD, acesso, auditoria.**

Redaction de verdade (em produção)

Lista de `paths` é frágil. Na **Kodus**, redação é por **nome de chave — em qualquer profundidade**:

- 🌳 **deepSanitize** — varre o objeto inteiro, não só `*.campo`
- abc normaliza caso/pontuação: `apiKey` = `api_key` = `API-KEY`
- 🪪 inclui `cpf`, `cvv`, `creditCard`, `jwt`, `connectionString` ...
- 🔗 **credencial em URL** → `mongodb://user:[REDACTED]@host`

“ `paths` pega o previsível. **deepSanitize** **pega o que vazaria.** ”

O ponto alto

Joga o log de erro pra uma IA...
...e ela acha o bug sozinha.

A IA achando o bug

Prompt: "Você é um SRE. Causa raiz desse log? Sugira o fix."

Resposta:

“ O desconto (9000) é maior que o subtotal (4990), então total_cents ficou **negativo** (-4010). O gateway rejeitou com INVALID_AMOUNT. Cliente **premium** afetado.
Fix: Math.max(0, subtotal - discount). ”

Achou um bug de **regra de negócio** sem ver o código.
Só foi possível porque o log é **estruturado e contextualizado**.

O bug — e o fix em 1 linha

✗ O que o log revelou (`total_cents: -4010`):

```
const total = cart.subtotalCents - discount; // fica negativo 🐛
```

✓ O fix:

```
const total = Math.max(0, cart.subtotalCents - discount);
```

A IA leu o evento, achou a causa e sugeriu isso.
Sem o log estruturado, nada disso acontece.

O que faz a IA entender o log

Os princípios que você já viu **são exatamente o que um LLM precisa**:

- **Sem memória entre linhas** → o wide event dá o caso **inteiro** de uma vez
- **Campos nomeados** → ela **deduz** a causa, em vez de chutar
- **trace_id** → reconstrói a requisição (log + trace) pra raciocinar
- **Sem PII** → redaction vira **fronteira**: a IA não vê o que não deve

“ Mesma IA, **deu erro** vs wide event: resultado completamente diferente. ”

O custo escondido: log pesa na app

```
for (const item of items) logger.info(item); // 🐛 log em loop quente
logger.info({ payload: bigJsonDeMBs });      // serializa MBs por request
logger.debug(montaStringCara());             // roda MESMO com debug off!
```

- **Síncrono no hot path** → derruba throughput (req/s)
- **Serializar objeto gigante/circular** → CPU + pressão de GC (memória)
- **throw pra fluxo normal + try/catch** → captura de stack é cara
- **Transport pesado em prod** (pretty/HTTP no processo) → compete com a app

“

Logar é I/O e CPU. Mal feito, seu observability **vira o gargalo**.

”

E os níveis? (debug, info, warn, error)

Nível	Quando usar
debug	detalhe pra investigar — desligado em produção
info	o fato normal: o wide event da requisição
warn	estranho, mas seguiu (retry, fallback, deprecation)
error	falhou de verdade — alguém precisa agir
fatal	o processo vai morrer

Na prática: viva em `info` + `error`. `debug` sob flag, `warn` com parcimônia.

Menos níveis = menos decisão = **schema mais consistente**.

Produção: o que importa de verdade

Tema	Decisão
Níveis	info + error . Só isso.
Volume	sampling: 100% erro, amostra de sucesso
Performance	JSON no stdout; coletor envia; nada de log em loop
Segurança	redaction de PII/secrets na origem
Retenção	TTL por classe; hot/cold; agrega em métrica
Custo	GB/dia é métrica de produto

Ferramentas: o código não muda

```
app → OTLP (ou stdout) → coletor → backend
```

Coletor: OpenTelemetry Collector · Vector · Fluent Bit

Logs: Loki · ELK/OpenSearch · Datadog · SigNoz · Axiom

Traces: Tempo · Jaeger · Zipkin

“ Começa barato e sem lock-in: **OTel Collector + Grafana (Loki+Tempo)** ”

Alertas + AI analytics

- **Alerte em sintoma** (taxa de erro, p95, SLO burn) — **não** em `logger.error`
- Wide event deixa o alerta **rico**: já sabe *quem* e *quanto*
- O mesmo evento vira **dataset**: pergunte em linguagem natural
 - *"quais clientes premium falharam no checkout hoje?"*
- Anomalia, triagem por impacto, correlação com `commit` — de graça

“ Mesmo dado, três consumos: **dashboard, alerta e IA.** ”

Troubleshooting: o playbook

1. **Alerta** dispara (sintoma)
2. **Filtra** `outcome:error AND path:/checkout`
3. **Lê** o contexto num evento (user, valor, cupom, commit)
4. **Pula pro trace** pelo `trace_id`
5. **Correlaciona** com deploy pelo `commit`
6. **Pergunta pra IA** → causa raiz + fix

Antes: 5 telas, 40 min. **Depois:** 1 query, 1 pergunta.

Pra levar pra casa

Estrutura.

Contexto.

Correlação.

O resto é ferramenta.

O próximo passo já começou: **log → IA acha a causa → IA abre o PR.**

Obrigado! 🙏

Bora trocar ideia — me acha aqui:

🔗 **LinkedIn** — [linkedin.com/in/wellington-santana-a48b1123](https://www.linkedin.com/in/wellington-santana-a48b1123)
✂ **Twitter/X** — @wellingtoncvs

Repo da demo: github.com/Wellington01/logs-que-a-ia-entende
Pino · OpenTelemetry · Grafana Loki + Tempo

Perguntas?